

Drawing VS Scheduling

Materials/Hardware Schedule Extraction & Cross-Drawing Validation

1. Objective

To automatically extract the materials and hardware schedule table from each sheet of an architectural drawing set, group sheets into consecutive drawing sets, and verify — without any manual cross-checking — that every code listed in a schedule table is actually referenced somewhere in that drawing set, and vice versa.

2. Problem Statement

Architectural submittals commonly repeat the same schedule table format across many sheets, while codes referenced in the drawings (via leader lines and callout bubbles) must match what each sheet's own table lists. Checking this by hand across a large sheet set is slow and error-prone. In addition, an identical schedule table can legitimately be reprinted across two unrelated drawing groups, so naive similarity-based grouping risks merging sheets that should stay separate. There is also no ready-made ground truth to confirm a code is genuinely missing versus simply drawn elsewhere on the same sheet.

3. Proposed Approach

We propose reading everything directly from the PDF's underlying vector and text data — no OCR — and using two independent, purpose-built signals: page order for grouping sheets into drawing sets, and PDF geometry (lines, text blocks, and shapes) for extracting and cross-checking schedule codes against what actually appears in the drawing area. Validation is performed at the group level, since a code and its drawing reference commonly sit on different sheets within the same set.

4. Methodology

The pipeline runs through eight phases, from raw sheet identification to a final structured validation report:

#	Phase	Description
1	Title & Sheet Identification	Read each page's title block to get its sheet ID, MOD number, and scope, and detect every drawing-view title on the page from scale/ARCH REF text anchors.
2	Drawing Grouping	Walk the PDF in page order and start a new group only when a main PLAN or KEY PLAN title appears; every following page joins that group until the next one starts.
3	Table Extraction	Locate each sheet's materials/hardware schedule table from the PDF's vector lines and text, and read out every code, shape, and description row by row.
4	Table Signatures	Compute a lightweight signature of each table's codes and content, kept for diagnostics and caching — not used to decide grouping.
5	Drawing-Area Verification	For every code found in a group's tables, search that group's drawing area (across all its sheets) to confirm the code is actually referenced there.

6	Group-Level Validation	Combine the two checks: does every scheduled code appear somewhere in the drawing, and does every leadered code appear in a table? Mark the group PASS or FAIL accordingly.
7	Per-View Section Reference Check	Separately, follow PLAN / PLAN SECTION callout bubbles to their target drawing and confirm the codes shown there are backed by a table on either the source or target sheet.
8	Reporting	Print a full group-by-group report to the console and write a single structured JSON file summarizing every group, code, and validation result.

4.1 Drawing Grouping — page order, not similarity

Sheets are grouped strictly by their position in the PDF: the first page always starts Group 1, and a later page starts a new group only when it carries a main PLAN or KEY PLAN title. Secondary plan-type titles — PLAN SECTION, RCP, Roof Plan, Ceiling Plan — do not start a new group, since these are typically continuation views within an already-open set. Table similarity and cross-reference bubbles are deliberately excluded from this decision, so a schedule table reprinted identically in two adjacent groups does not cause those groups to merge.

4.2 Table Extraction from PDF Geometry

Each sheet's schedule table is located by scoring clusters of horizontal ruling lines against nearby header text (MATERIALS, HARDWARE, DESCRIPTION, SCHEDULE, CODE, SHAPE, QTY). Once the table region is found, each row's code, shape marker, and description are read out and normalized — including correcting common OCR-style prefix errors and restoring decimal points in multi-part codes — then checked against an approved code-format allow-list.

4.3 Group-Level Drawing Verification

Every code collected from a group's tables is searched for across the drawing area of every sheet in that group, using the code's associated shape marker (rectangle, octagon, or circle) as an additional confirmation signal. A code is marked orphaned if it is scheduled but never found in any drawing in the group; a schedule omission is logged if a code is leadered on a sheet's drawing but missing from that specific sheet's own table. A group is only marked PASS when neither condition occurs.

4.4 Per-View Section Reference Validation (additive check)

As a separate, additive validation pass, each PLAN or PLAN SECTION view's callout bubble is resolved to its target sheet and view. The codes shown in the target drawing are then checked against both the target sheet's own table and the source sheet's table, flagging any code that appears in neither.

5. Key Design Decisions

Decision	Rationale
Grouping is page-order only	A new group starts strictly on a main PLAN/KEY PLAN title. Table similarity and cross-reference bubbles are deliberately excluded from this decision, so an identical table reprinted in two adjacent groups no longer merges them.
No OCR is used	All text and layout comes directly from the PDF's own vector and text data (via PyMuPDF), which is faster and more reliable than image-based recognition for born-digital drawing sheets.

Codes are allow-listed	Only H-<numeric> hardware codes and the approved material/finish prefixes (MM, PL, WD, WV, PT, GL, SS, LAM) are accepted as valid schedule codes, rejecting anything outside the approved project format.
Validation runs at group level	A code is checked against the combined drawing area of every sheet in its group, not just the single sheet its table happens to sit on — matching how these drawings are actually organized and cross-referenced in practice.
Failure has two independent causes	A group fails if it has an orphaned code (scheduled but never drawn) OR a schedule omission (drawn but missing from that sheet's own table) — either condition alone is treated as a real defect.

6. Usage

The script is run from the command line against a single PDF submittal:

```
python table_extractor3.py <input.pdf> [--out results.json] [--quiet]
```

- `input_file` — path to the PDF drawing set to process
- `--out / -o` — path to write the structured JSON validation output (default: `elevation_groups_validation.json`)
- `--quiet / -q` — suppress per-match debug lines while keeping the full report

7. Deliverables

- A console report broken down group by group: sheets, view titles, extracted codes, code presence across sheets, and drawing verification results
- A per-view section-reference validation report tracing PLAN/PLAN SECTION callouts to their target drawings
- A single structured JSON file containing every group's codes, PASS/FAIL status, orphaned codes, and schedule omissions, ready for downstream reporting or QA tooling
- A final summary of total groups, total sheets, and overall PASS/FAIL counts across the submittal

8. Conclusion

This approach relies entirely on the PDF's own vector and text data rather than OCR or image recognition, keeping extraction fast and precise on born-digital drawing sets. By separating drawing grouping (page order) from schedule validation (PDF geometry and code matching), and by validating at the group level rather than sheet by sheet, the pipeline reflects how these submittals are actually organized in practice — while remaining resistant to the false merges that similarity-based grouping would otherwise introduce.